Architectural Strategies for Resolving Linked Identity Identifier Disparity in Evolution API and N8N Integrations

The rapid transformation of the WhatsApp protocol architecture, specifically the transition from phone-number-based identifiers to abstract session-based identifiers, has created a significant technical hurdle for developers using the Evolution API as a bridge for automation. At the core of this challenge is the emergence of the Linked Identity (LID) identifier, which replaces the traditional @s.whatsapp.net Jabber ID (JID) in various message events and webhook payloads. For organizations utilizing n8n for workflow orchestration, this shift necessitates a fundamental rethinking of how users are identified, indexed, and engaged within automated systems. The following report provides a comprehensive technical analysis of the LID mechanism, the specific data fields available for resolution within the Evolution API, and the architectural strategies developed by the community to maintain reliable user tracking.

The Evolution of WhatsApp Identifiers and the Linked Identity Mechanism

To understand the current state of identifier resolution, one must first examine the historical context of the WhatsApp protocol. Historically, the Jabber ID (JID) was a direct reflection of the user's telecommunications credential, formatted as [country_code][phone_number]@s.whatsapp.net. This immutable link allowed developers to use the phone number as a primary key across CRM systems, databases, and messaging platforms without the need for an intermediate translation layer. However, the rollout of multi-device support and enhanced privacy features led Meta to introduce the @lid (Linked Identity) suffix.[1]

The @lid identifier serves as an internal, platform-specific surrogate that enables WhatsApp to manage multiple concurrent sessions across different devices (e.g., a primary smartphone, a desktop client, and a tablet) without consistently exposing the primary phone number in every message packet. This transition is particularly prevalent among Android users and in scenarios involving new, unauthenticated contacts.[3] The shift from a phone-based JID to a numeric string followed by @lid (for instance, 4252168646732@lid) effectively breaks legacy lookup mechanisms that depend on the @s.whatsapp.net format for user identification.[4]

The implications for automation are profound. When an n8n workflow receives a webhook from the Evolution API, the primary identification field, remoteJid, may no longer contain the phone number. If the workflow attempts to query a database or an application like Airtable using the phone number, the query will fail to return a match for the @lid string, leading to duplicate records, fragmented conversation histories, and a general failure of the automation logic.[4]

Technical Dissection of Evolution API Webhook Payloads for User Identification

The Evolution API developers have introduced several mechanisms to mitigate the impact of the LID transition. While the remoteJid field is increasingly populated with the LID, the underlying Baileys library and the Evolution API's message processor attempt to capture and expose the original phone-based identifier in parallel fields within the webhook JSON payload.

Analysis of the RemoteJidAlt and SenderPn Fields

The most critical field for developers seeking to maintain phone-based identification is remoteJidAlt. In Evolution API version 2.x and subsequent releases, when the platform detects a message originating from an identity using the LID format, it attempts to populate remoteJidAlt with the normalized, phone-number-based JID, such as 5511999999999@s.whatsapp.net.[6] This field acts as a secondary identifier that preserves the link to the user's primary account, even when the remoteJid primary field reflects the temporary or session-based LID.

In certain interaction flows, the senderPn (Sender Phone Number) field may also be available. This field explicitly contains the raw phone number string, providing a fallback for manual JID construction in n8n workflows if remoteJidAlt is absent.[1] The availability and reliability of these fields can vary based on the sender's device type and the specific version of the WhatsApp client they are using.

Field Name	Typical Format	Reliability for Identification	Functional Role
remoteJid	[string]@lid or [phone]@s.whatsapp.net	Moderate	The primary session identifier for the current message. [1]
remoteJidAlt	[phone]@s.whatsapp.net	High	The alternative, normalized identifier mapping back to the phone number. [6]
senderPn	[phone_number]	Moderate	The raw phone number of the sender, if exposed by the protocol layer. [1]
participant	[string]@lid	Low	Often mirrors the LID in group contexts, complicating individual identification. [5]

For n8n users, the remoteJidAlt field represents the most robust option for cross-referencing. Community documentation and official pull requests indicate that even when a message

arrives with a remoteJid ending in @lid, the remoteJidAlt will frequently contain the expected @s.whatsapp.net JID, allowing the automation to query external databases without modifying the underlying schema.[6]

The Impact on N8N Integration and Logic Normalization

For n8n developers, the presence of @lid identifiers requires an explicit normalization layer at the entry point of any workflow. If the workflow expects a standard phone number but receives an LID, subsequent nodes that filter or search based on the phone number will fail. The most effective method for handling this involves using a ternary expression within a "Set" node or an "HTTP Request" node to dynamically select the correct identifier.
The following logic is commonly used to ensure that the workflow consistently operates on the @s.whatsapp.net format:

JavaScript

```
{{ ( $json.body.data.key.remoteJid.endsWith('@lid')? $json.body.data.key.remoteJidAlt : $json.body.data.key.remoteJid ).toString().replace(/[\n\r\t ]+/g, '') }}
```

This expression evaluates the suffix of the remoteJid. If it ends in @lid, it switches to the remoteJidAlt field. If not, it proceeds with the standard remoteJid. This ensures that regardless of whether the message originates from a legacy device or a newer multi-device session, the identity remains stable within the automation.[6]

API Endpoints for Identifier Resolution and Mapping

In scenarios where the webhook payload arrives without a populated remoteJidAlt field—which can occur with first-time contacts or due to specific privacy settings on the sender's device—the Evolution API offers dedicated endpoints to perform manual resolution. These endpoints query the internal cache and the WhatsApp protocol to retrieve the contact profile and any associated phone numbers.

The Contact Profile and FetchContacts Endpoints

The GET /contact/profile/{instance}/{remoteJid} endpoint is the primary tool for resolving an LID into a full profile object. When called with an @lid identifier, the Evolution API attempts to fetch the latest profile data from the WhatsApp servers. This returned JSON object frequently includes the actual phone number, allowing an n8n workflow to make an "on-the-fly" API call to resolve the identifier before proceeding to database operations.[2]
Alternatively, the GET /contacts/fetchContacts endpoint provides a comprehensive list of all contacts stored in the instance's cache. Analysis indicates that this endpoint returns objects containing both the LID and the actual phone number in a unified structure.[4] This is particularly useful for building a local mapping table, which can serve as a high-performance lookup cache to avoid repeated external API calls.

Comparative Analysis of Resolution Endpoints

The following table compares the primary endpoints used for identifier resolution and mapping:

Endpoint Path	HTTP Method	Required Parameters	Primary Use Case
/contact/profile	GET	instance, remoteJid	Immediate, single-user resolution when remoteJidAlt is missing. [2]
/contacts/fetchContacts	GET	instance	Bulk synchronization of LID-to-phone mappings for local caching. [4]
/contacts/findContacts	GET	where filters	Searching for a contact record when only partial data (like pushName) is known. [10]
/chat/getBase64FromMedia	POST	messageKey	Retrieving media associated with a message when database storage is disabled. [4]

The architecture of these endpoints reflects the Evolution API's role as a sophisticated state manager. By utilizing /contacts/fetchContacts, developers can implement a "Lazy Loading" pattern in n8n: if an incoming LID is not found in the local database (such as Airtable or Postgres), the workflow triggers an API fetch to update the mapping and then resumes the main automation path with the resolved phone number.[4]

Configuration Management and Environment-Level Overrides

The Evolution API allows for certain behaviors to be toggled at the instance or global environment level. This is crucial for developers who wish to minimize the exposure of @lid identifiers or ensure that the API prioritizes phone-based JIDs in its internal logic.

Disabling LID Mode and Number Check Configurations

A significant configuration option available in the Evolution API's Docker environment is the WPP_LID_MODE variable. Setting WPP_LID_MODE=false instructs the underlying WhatsApp protocol handler to attempt to suppress LID identifiers in favor of the standard JID format.[2] While this may not be successful in every case—as Meta occasionally enforces LID-based sessions regardless of client-side preferences—it provides a strong directive to the API to

maintain legacy identification patterns where possible.
Furthermore, developers often ask if there is a way to disable "number checks" to allow sending messages directly to an LID without a phone number. In Evolution API v2.x, the platform has been updated to support @lid as a valid recipient identifier in the /message/sendText and other messaging endpoints.[10] This means that if an application captures an LID from an incoming webhook, it can respond directly to that identifier without ever needing to resolve the phone number. The API handles the routing internally, ensuring that the message reaches the correct user session.[10]

Database Persistence and Contact Synchronization

The settings related to database persistence also influence how identifiers are stored and retrieved. The DATABASE_SAVE_DATA_CONTACTS flag ensures that the API maintains a local record of resolved phone numbers in the specified database (PostgreSQL, MySQL, or Redis). This makes subsequent lookups via the API endpoints faster and more reliable, as the mapping is already present in the local cache.[10]

Environment Variable	Recommended Value	Impact on LID Handling
WPP_LID_MODE	false	Attempts to force the use of standard phone-based JIDs over LIDs. [2]
DATABASE_SAVE_DATA_CONTACTS	true	Ensures that the mapping between LIDs and phone numbers is persisted in the database. [13]
DATABASE_SAVE_IS_ON_WHATSAPP	true	Caches the "Is on WhatsApp" check results, reducing protocol overhead. [13]
CHATWOOT_BOT_CONTACT	false	Prevents the creation of duplicate bot contacts in Chatwoot integrations. [14]

By correctly configuring these variables, an organization can create a "Self-Resolving" environment where the Evolution API handles much of the complexity of the LID transition before the data even reaches n8n.

Community-Developed Workarounds and Best Practices

The transition to @lid has fostered a robust ecosystem of community-developed solutions. These strategies generally fall into two categories: real-time normalization and persistent mapping tables.

Real-Time Normalization via N8N Workflow Nodes

For many developers, the most pragmatic solution is to insert a "normalization" step at the beginning of every n8n workflow triggered by the Evolution API. This step involves a "Filter" node to detect if the remoteJid ends with @lid. If it does, the workflow calls the remoteJidAlt field or, if necessary, the /contact/profile endpoint to resolve the identity.[2] This adds a small amount of latency to the execution but ensures that all subsequent nodes (e.g., CRM lookups, order processing) receive a reliable phone number.

This method is particularly effective for businesses that do not have a massive volume of new contacts, as the API overhead remains manageable. The workflow typically follows this logic:

1. **Webhook Node**: Receives the message from Evolution API.
2. **Switch Node**: Checks the suffix of data.key.remoteJid.
3. **HTTP Request Node (Optional)**: If remoteJidAlt is missing, call /contact/profile to resolve the LID.
4. **Set Node**: Unifies the data so that the resolved number is placed in a standard user_id variable for the rest of the workflow.

Persistent Mapping Tables for High-Volume Applications

For high-volume enterprise applications, the latency and potential rate-limiting of real-time API calls make resolution-on-the-fly less desirable. Instead, developers often implement a persistent mapping table in their database, such as Airtable or PostgreSQL.[4] This table acts as an "Identifier Translation Layer," storing the relationship between LIDs and phone numbers as they are discovered.

When a message arrives, the system queries this table first. If the LID is known, the phone number is retrieved instantly. If the LID is unknown, the system performs a one-time resolution via the Evolution API, updates the table, and continues. This architectural pattern provides the highest level of stability and performance, as it minimizes the number of external protocol-level requests.[4]

Architectural Challenges with Landline and Business Numbers

Further complicating the identification landscape is the behavior of landline numbers or WhatsApp Business accounts verified via voice call. Reports indicate that instances connected to landlines may experience issues with webhook triggers or inconsistent LID reporting.[15] This suggests that the Meta protocol handles Business-verified landlines differently than standard mobile accounts, potentially requiring more aggressive caching of contact profiles.

When working with landlines, it is recommended to explicitly set DATABASE_SAVE_DATA_INSTANCE=true and DATABASE_SAVE_DATA_CONTACTS=true to ensure that the Evolution API has sufficient local context to reconstruct the JID relationships.[13] Additionally, the use of syncFullHistory=true in the instance settings can help populate the contact cache with historical mappings before the first active message is received.[10]

Integration with Third-Party Platforms: Typebot and Chatwoot

The Evolution API serves as a middleware for several popular open-source platforms, and the way these tools handle LIDs provides insight into how the API's internal logic has evolved to meet industrial demands.

Typebot and LID-Aware Routing

Recent updates to the Evolution API (v2.3.7 and later) have specifically addressed how Typebot interacts with LID identifiers. Typebot has been updated to maintain the complete JID—including the @lid suffix—when responding to messages, rather than attempting to extract only a numeric portion.[11] This change ensures that the chatbot's response is correctly routed back to the specific session that initiated the conversation.

The fix involves a conditional check within the message routing logic: remoteJid.includes('@lid')? remoteJid : remoteJid.split('@'). This pattern allows the system to be "format-agnostic," supporting both the traditional phone-based JID and the newer LID format without breaking the logical flow of the conversation.[10]

Chatwoot and Contact De-duplication Strategies

Chatwoot integrations often face the challenge of duplicate contact creation when an LID is received instead of a phone number. If the Evolution API does not resolve the LID before sending the data to Chatwoot, the help desk platform may create a new contact with the LID as the name or identifier.[3]

To prevent this, it is essential to ensure that the Evolution API version is updated to a release that includes "LID normalization" in the message upsert path. These versions automatically mutate the remoteJid to the remoteJidAlt before the webhook is emitted to Chatwoot, ensuring that the help desk platform receives the stable phone-based JID.[10] This prevents the fragmentation of customer support tickets and ensures a unified view of the customer's history.

Future Outlook and Strategic Strategy for Self-Hosted WhatsApp Bridging

The transition to LID identifiers is not a temporary anomaly but a foundational shift in how WhatsApp manages identity within its multi-device ecosystem. As Meta continues to move toward a unified messaging infrastructure across WhatsApp, Instagram, and Messenger, the reliance on a physical phone number as the sole identifier will likely continue to diminish in favor of abstract, session-based tokens.

For developers, the long-term strategy must prioritize identifier agnosticism. This involves designing database schemas that can handle multiple identifier formats for a single user record and utilizing the Evolution API's internal database (PostgreSQL/Redis) to maintain a persistent

link between session identifiers (LID) and permanent identifiers (Phone). By embracing these patterns and utilizing proactive resolution via the /contacts/fetchContacts endpoint, organizations can build resilient, enterprise-grade automation that remains stable despite the evolving nature of the WhatsApp protocol.

Conclusions and Actionable Recommendations

The management of LID identifiers in the Evolution API requires a multi-layered approach combining environment configuration, webhook payload parsing, and strategic API calls. The most reliable way to identify users by their phone number when an @lid message arrives is to leverage the remoteJidAlt field in the webhook payload, as it consistently contains the normalized @s.whatsapp.net JID. In cases where this field is absent, the GET /contact/profile endpoint remains the definitive source for identity resolution.

To optimize n8n workflows, developers should implement a normalization logic layer that prioritizes remoteJidAlt and uses a mapping table (such as Airtable or a local database) to cache resolved identities. This avoids the need for a secondary registration process and ensures compatibility with existing business systems. Furthermore, setting the WPP_LID_MODE=false environment variable and ensuring that DATABASE_SAVE_DATA_CONTACTS is enabled will provide a more stable foundation for user tracking. As the WhatsApp platform continues to evolve, maintaining a flexible, multi-identifier approach will be essential for the success of any messaging-based automation strategy.

Works cited

1. [META] @lid vs @jid handling — tracking umbrella · Issue #1872 ..., accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/issues/1872
2. Problema con Evolution API 1.6.1 ⟶ @lid en vez de número real - n8n Community, accessed on May 6, 2026, https://community.n8n.io/t/problema-con-evolution-api-1-6-1-lid-en-vez-de-numero-real/270857
3. [BUG] remoteJid is differenten than the real whatsaap number · Issue #1916 · EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/issues/1916
4. Evolution API v2 + n8n: @lid remoteJids don't contain phone numbers — anyone solved this? - Reddit, accessed on May 6, 2026, https://www.reddit.com/r/n8n/comments/1rocs4y/evolution_api_v2_n8n_lid_remotejids_dont_contain/
5. User Phone Numbers in participant Field Are Now Encrypted/Hashed · Issue #2035 · EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/issues/2035
6. [BUG] Erro no número do WhatsApp e pushName quando o evento vem de anúncio · Issue #2267 · EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/issues/2267
7. [BUG] Evolution-Api 2.3.6 Not Working Send-Text not working on some phone numbers · Issue #2272 - GitHub, accessed on May 6, 2026,

https://github.com/EvolutionAPI/evolution-api/issues/2272
8. [NOWEB] - I cant relate some lids to the contact phone number · Issue #1097 · devlikeapro/waha - GitHub, accessed on May 6, 2026, https://github.com/devlikeapro/waha/issues/1097
9. Typebot integration sends lid and not jid · Issue #2251 · EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/issues/2251
10. Releases · EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/releases
11. EvolutionAPI/evolution-api 2.3.7 on GitHub - NewReleases.io, accessed on May 6, 2026, https://newreleases.io/project/github/EvolutionAPI/evolution-api/release/2.3.7
12. Send Text | Evolution API | v2.0 - Postman, accessed on May 6, 2026, https://www.postman.com/agenciadgcode/evolution-api/request/gsjxjzm/send-text
13. docs.squarecloud.app, accessed on May 6, 2026, https://docs.squarecloud.app/llms-full.txt
14. CHANGELOG.md - EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/blob/main/CHANGELOG.md
15. Evolution API webhook works with cell phone number, but not with landline number (WhatsApp Business) : r/n8n - Reddit, accessed on May 6, 2026, https://www.reddit.com/r/n8n/comments/1r9fql5/evolution_api_webhook_funciona_com_n%C3%BAmero_celular/?tl=en
16. Set Settings - Evolution API Documentation, accessed on May 6, 2026, https://docs.evoapicloud.com/api-reference/settings/set
17. fix: improve type safety by removing 'as any' casts by jeandgardany · Pull Request #2451 · EvolutionAPI/evolution-api - GitHub, accessed on May 6, 2026, https://github.com/EvolutionAPI/evolution-api/pull/2451